Project 4 Single Sensor PID Line Follower

Omar Zid

Informatics in Robotics | 14 October 2026

Simulation study. All numerical measurements are generated by the supplied software models. They are not measurements from a physical LEGO NXT robot. Hardware behavior requires calibration and testing.

Omar Zid | Simulated PID completed one marker-to-marker lap in 20.880 s. Sensor-edge RMS error was 0.639 mm, compared with 1.712 mm for P and 0.782 mm for PI. Wheel dynamics, saturation, noise, bias and a repeatable heading disturbance are included.

Assignment source: Screenshot_7.png

Task track geometry and one sensor observation

All trials in this report are reproducible software simulations produced for Omar Zid. They are not physical measurements, proof of NXC compilation, or evidence of testing a LEGO robot. Motor B is the left wheel; motor C is the right wheel; the one color/light sensor is S1 and the fixed forward ultrasonic sensor is S4. Wheel diameter D = 56 mm and wheel-center track B = 120 mm are explicit assumed calibration values. Replace them with measured values before a physical demonstration.

The assignment requires a robot with only one color sensor to follow a line for one round using PID regulation. The software test uses a circular black line of centerline radius 350 mm and width 20 mm, so its inner and outer edges are radii 340 and 360 mm. The sensor is 35 mm ahead of the axle. The robot follows the outer edge at target reflectance 50, between calibrated black 20 and white 80. A smooth 2 mm edge transition models the sensor footprint. The reading has Gaussian noise $\sigma = 1.2$ reflectance units and a slow ±1.2-unit lighting drift.

A broad 18 mm transverse black marker crosses the normal line once per circuit. The starting pose aligns the same forward sensor with this marker. Initial marker readings cannot end the run. Lap detection is armed only after encoder path exceeds 1500 mm; after arming, ten consecutive 10 ms readings below 25 trigger braking. The marker is wider than normal transient dark readings and its expected dwell is about 0.164 s at 110 mm/s. No second color sensor or hidden ground-truth track angle controls termination.

The independent checker unwraps the true polar angle of the axle center and verifies that angular travel is at least 2π rad. That measurement validates the result after the controller stops; it is not supplied to PID or the stop condition. The start-marker alignment matters because a forward sensor crosses the finish before the axle center reaches the same position. Starting with the sensor on the marker makes the next encounter a full sensor-to-sensor circuit.

Time discrete PID saturation and anti windup

Error is $e = (\text{reading} - 50)/30$, a dimensionless normalized reflectance difference. Positive error means the outer-edge sensor is too far onto white; the command turns left to return toward black. Integral state I accumulates $e\Delta t$ in seconds, and the derivative has units s^{-1}. A first-order filtered derivative is $d[k] = 0.80d[k-1] + 0.20(e[k] - e[k-1])/\Delta t$. This reduces differentiation of sample noise.

The PID command is $\omega = K_p e + K_i I + K_d d$, in rad/s. Integral state is clamped to ±2 s, and conditional integration rejects a proposed update if the command is saturated and the current error would drive it further

into saturation. During the broad marker, the previous angular command is held and integration is frozen so the marker does not produce a steering kick. At startup the known entry command is 0.31 rad/s while the sensor leaves the marker.

Base translational command is 110 mm/s. Wheel commands are vL = 110 - 60ω and vR = 110 + 60ω mm/s. Angular command is limited to ±1.1667 rad/s, and wheel commands to -50-180 mm/s. Each motor follows a first-order lag τ = 0.10 s. The right motor produces 97% of its command, introducing a repeatable speed bias. At t = 8.00 s an external 8° heading disturbance is injected into the true pose; encoder heading does not observe this bump. The color feedback must recover.

Controller	Kp (s^{-1})	Ki (s^{-2})	Kd	Sample period (s)	Filter
P	0.85	0.0	0.0	0.01	0.80 / 0.20
PI	0.7	0.28	0.0	0.01	0.80 / 0.20
PID	0.8	0.25	0.035	0.01	0.80 / 0.20

Worked geometry and gain unit calculations

Radians are treated as dimensionless in the unit table: Kp produces rad/s per normalized error, Ki rad/s² per normalized accumulated error, and Kd rad per normalized derivative. These numerical gains are valid for the defined normalization, 10 ms loop and wheel-speed mapping; copying them into a raw 0-100 reflectance or motor-percentage loop changes the controller.

Tracking error is the actual sensor-point radius minus the outer edge radius, in mm. It is computed from the independent simulated pose for evaluation. PID itself receives only reflectance. Because the sensor lies ahead of the axle, axle radius 360 mm is not the zero sensor-error trajectory.

$$Zero - error\ axle\ radius\ = \sqrt{(360^2\ -\ 35^2)}\ =\ 358.295\ mm.$$

Desired circular angular rate at nominal speed ≈ 110/358.295 = 0.3070 rad/s. Corresponding wheel speeds are 110 ± 60 × 0.3070 = 91.58 and 128.42 mm/s.

For reading 56: e = (56 - 50)/30 = 0.20. If I = 0.40 s and filtered derivative d = 0.50 s^{-1}, PID command = 0.80 × 0.20 + 0.25 × 0.40 + 0.035 × 0.50 = 0.2775 rad/s.

For that worked command, vL = 110 - 60 × 0.2775 = 93.35 mm/s and vR = 126.65 mm/s. Encoder speeds are 191.02 and 259.16 degrees/s respectively for 56 mm wheels.

Angular saturation = (180 - 110)/(120/2) = 1.1667 rad/s. Integral clamp $|I| \le 2$ s prevents an integral term larger than 0.25 × 2 = 0.50 rad/s in PID.

Expected marker dwell = 18 mm/110 mm/s ≈ 0.164 s. Required dark confirmation = 10 × 0.010 = 0.100 s, leaving about 64 ms nominal margin.

Reference axle-circle circumference = 2π × 358.295 = 2251.23 mm; actual route is measured from wheel increments and varies with tracking error, startup and stop location.

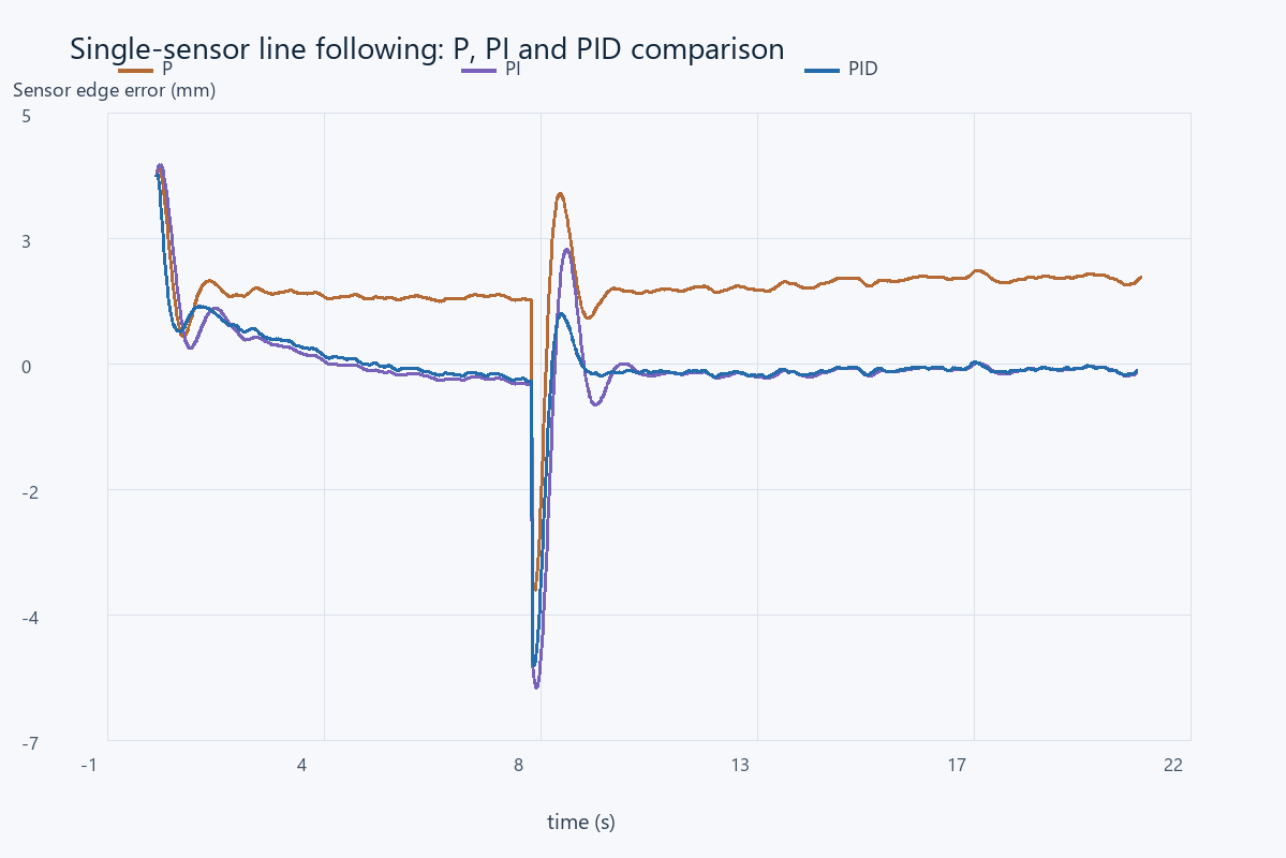
P PI and PID executed comparison

Each comparison uses the same starting geometry, random seed, wheel lag, 3% right-motor command bias, lighting drift and 8° heading disturbance. Only controller gains change. Error statistics include samples after the

first second and exclude the broad marker, where the sensor reading intentionally represents the finish stripe instead of the normal edge. Maximum error is the largest absolute sensor-edge error in that evaluation interval.

P requires a steady reflectance error to create the sustained turn and counter the motor bias. PI reduces that steady error but its integral responds more slowly to a sudden heading disturbance. PID adds a filtered derivative and achieved the smallest RMS error in this selected scene. Its maximum transient error can still exceed P even when its overall RMS is lower; a low RMS is not a guarantee of minimum peak error. These gains are compared within one specified track, not claimed optimal for every path.

Mode	Lap time (s)	RMS (mm)	Peak (mm)	Mean (mm)	Saturated samples
P	20.960	1.712	3.890	1.605	0
PI	20.880	0.782	5.674	0.123	0
PID	20.880	0.639	5.270	0.176	6



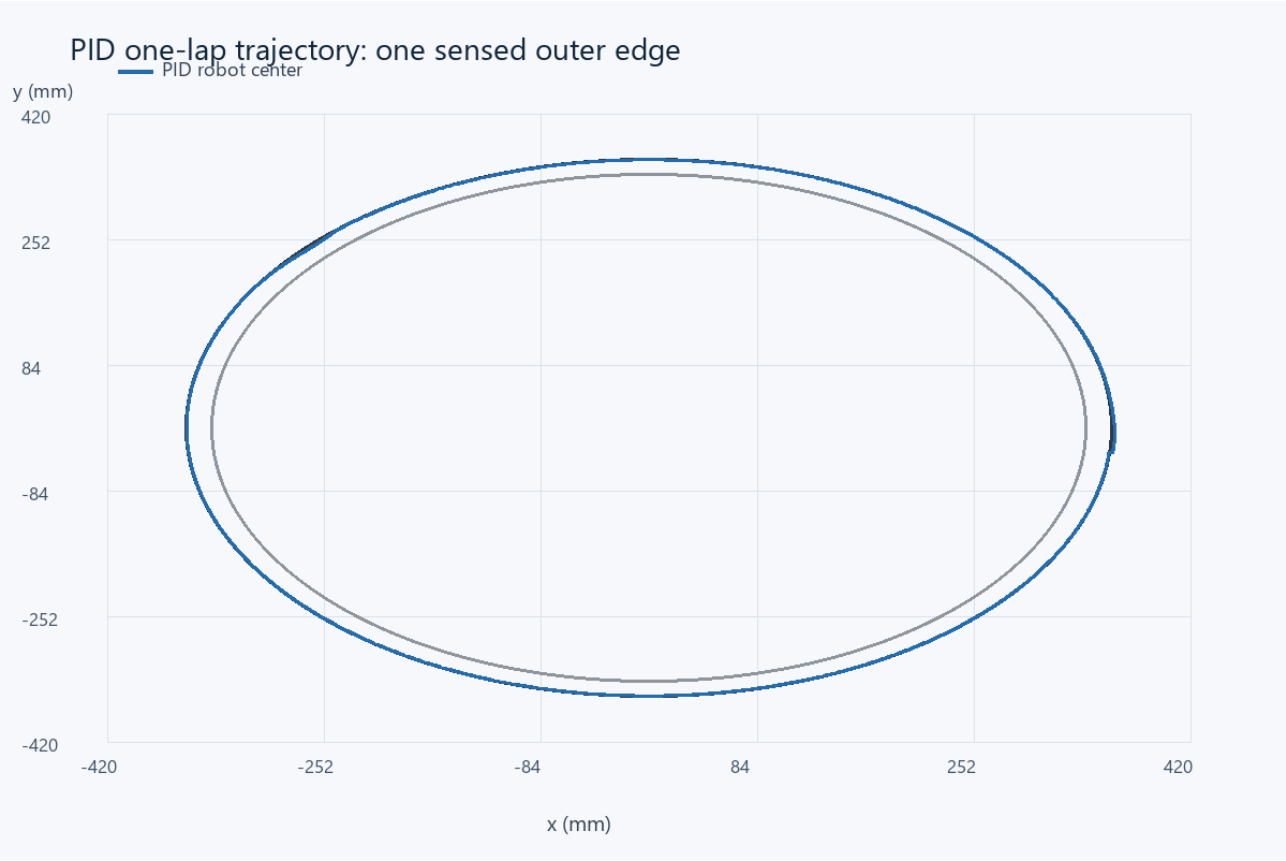
Computed sensor-edge error over time. The external heading disturbance occurs at 8.00 s. Marker samples are omitted from these error curves.

One lap evidence trajectory and trace

PID stopped after 20.880 s and 2256.36 mm of integrated wheel travel. Independent axle angular travel was 6.29179 rad, at least 2π = 6.28319 rad. The controller used its armed single-sensor marker detector for this stop. It did not stop after a fixed guessed lap time or when a precomputed angle became 360°.

The full p4_pid_trace.csv contains time, robot and encoder pose, wheel degrees, reflectance, normalized error, integral, angular command, both actual wheel speeds, saturation flags, marker flags, sensor-edge error and independent lap angle. The P and PI traces have the same schema for checking the comparison. Ground-truth error and lap angle are evaluation columns; they are not feedback inputs.

A unique finish marker and a simple track without branches or shortcuts are assumptions of this one-sensor method. Encoder arming rejects the initial marker but cannot mathematically prove a full lap on any unknown track. For a course with repeated black patches, intersections or a shorter loop, the arming distance and marker design must be changed and independently checked.



Actual PID axle-center trajectory, with the inner and outer black-stripe edges drawn at 340 and 360 mm radius.

Hardware tuning and model limits

For physical use, mount the one sensor 35 mm ahead at consistent height, measure black/white calibration, set the sensor on the intended edge, and confirm that positive correction turns toward black. Start with low forward power and Ki = Kd = 0; increase Kp until tracking is responsive, then add a small derivative to reduce overshoot and a small integral only if sustained bias remains. Keep the loop period fixed and include the real Δt in integral and derivative calculations. Motor percentages require measured speed conversion before these mm/s gains can be reused.

The simulation includes realistic control complications but remains a planar model: it omits stiction, battery changes, cable drag, uneven floors, sensor spectral behavior, wheel slip and noncircular sharp track corners. Braking at the finish is idealized. The intentional heading bump demonstrates recovery in this model and is not a measured bump test on hardware. A real lap should be timed separately, its sensor log saved, and its

completion observed by the teacher. The computed gains and traces supply a reproducible starting point rather than physical performance certification.

References

Assignment image: Screenshot_7.png - Project 4 LineFollower (6h).

Supplied NXC tutorial and NXC minimum reference, distributed with the source files.

Assignment-supplied PID help link:
http://www.inpharmix.com/jps/PID_Controller_For_Lego_Mindstorms_Robots.html (reference link reproduced from the brief; not a measured hardware source).

Files for reproducing and demonstrating the result

NXC source: 06_project4_pid_line.nxc. Shared configuration: common_config.nxc.

Open Start_Here.html in the package for visual results and file links. Run the scripts in the simulation folder to regenerate the data and checks. The code folder contains the NXC source program and hardware configuration notes. The simulation implements a separate software model; it does not execute the NXC binary.